

RAG-Contract-Demo — Standalone Local Demo

This canvas contains a complete, ready-to-run local demo project for the task your sir gave: ingest up to 5 contracts, build a RAG index (Qdrant), use a local Mistral model via Ollama for QA and "Generate Contract" flows, and a tiny React UI to interact with it.

Structure

```
rag-contract-demo/  
├─ backend/  
│   ├── ingest_contracts.py  
│   ├── rag.py  
│   ├── app.py  
│   └─ requirements.txt  
├─ frontend/  
│   ├── package.json  
│   └─ src/GenerateContract.jsx  
└─ README.md
```

backend/requirements.txt

```
fastapi  
uvicorn  
python-multipart  
qdrant-client  
sentence-transformers  
transformers  
docx2txt  
PyPDF2  
requests  
python-docx
```

backend/ingest_contracts.py

```
# ingest_contracts.py  
import os  
from pathlib import Path  
from sentence_transformers import SentenceTransformer  
from qdrant_client import QdrantClient  
from qdrant_client.http import models as qmodels  
import uuid  
import docx2txt
```

```

import PyPDF2

MODEL_NAME = "all-MiniLM-L6-v2"
VECTOR_DIM = 384
QDRANT_URL = os.environ.get("QDRANT_URL", "http://localhost:6333")
COLLECTION_NAME = os.environ.get("QDRANT_COLLECTION", "contracts")
CHUNK_SIZE = 800
CHUNK_OVERLAP = 200

def extract_text_from_pdf(path: str) -> str:
    text = []
    with open(path, "rb") as f:
        reader = PyPDF2.PdfReader(f)
        for p in reader.pages:
            text.append(p.extract_text() or "")
    return "\n".join(text)

def extract_text_from_docx(path: str) -> str:
    return docx2txt.process(path)

def chunk_text(text: str, size=CHUNK_SIZE, overlap=CHUNK_OVERLAP):
    chunks = []
    start = 0
    n = len(text)
    while start < n:
        end = min(start + size, n)
        chunk = text[start:end]
        chunks.append(chunk)
        # move start forward with overlap
        if end >= n:
            break
        start = end - overlap if end - overlap > start else end
    return chunks

def ingest_folder(input_folder: str = "uploaded_contracts"):
    model = SentenceTransformer(MODEL_NAME)
    client = QdrantClient(url=QDRANT_URL)

    # create or recreate collection if missing
    try:
        client.get_collection(COLLECTION_NAME)
    except Exception:
        client.recreate_collection(
            collection_name=COLLECTION_NAME,
            vectors_config=qmodels.VectorParams(size=VECTOR_DIM,
            distance=qmodels.Distance.COSINE),
            replication=1,

```

```

    )

ids = []
vectors = []
payloads = []

for file in Path(input_folder).iterdir():
    if file.suffix.lower() not in [".pdf", ".docx"]:
        continue
    if file.suffix.lower() == ".pdf":
        text = extract_text_from_pdf(str(file))
    else:
        text = extract_text_from_docx(str(file))
    if not text.strip():
        continue
    chunks = chunk_text(text)
    for i, chunk in enumerate(chunks):
        emb = model.encode(chunk).tolist()
        _id = str(uuid.uuid4())
        ids.append(_id)
        vectors.append(emb)
        payloads.append({
            "filename": file.name,
            "chunk_index": i,
            "text": chunk[:2000]
        })

if not ids:
    print("No vectors to upsert.")
    return 0

client.upsert(
    collection_name=COLLECTION_NAME,
    points=qmodels.Batch(
        ids=ids,
        vectors=vectors,
        payloads=payloads
    )
)

print(f"Upserted {len(ids)} vectors to '{COLLECTION_NAME}'")
return len(ids)

if __name__ == "__main__":
    cnt = ingest_folder()
    print("Done. Vectors upserted:", cnt)

```

backend/rag.py

```
# rag.py
import os
import requests
from qdrant_client import QdrantClient
from sentence_transformers import SentenceTransformer

QDRANT_URL = os.environ.get("QDRANT_URL", "http://localhost:6333")
COLLECTION = os.environ.get("QDRANT_COLLECTION", "contracts")
EMBED_MODEL = "all-MiniLM-L6-v2"
OLLAMA_API = os.environ.get("OLLAMA_API", "http://localhost:11434/api/generate")
OLLAMA_MODEL = os.environ.get("OLLAMA_MODEL", "mistral")

client = QdrantClient(url=QDRANT_URL)
embedder = SentenceTransformer(EMBED_MODEL)

def retrieve_contexts(query: str, top_k: int = 6):
    q_emb = embedder.encode(query).tolist()
    hits = client.search(
        collection_name=COLLECTION,
        query_vector=q_emb,
        limit=top_k,
    )
    contexts = []
    for h in hits:
        payload = h.payload
        contexts.append({
            "score": getattr(h, 'score', None),
            "text": payload.get("text", ""),
            "filename": payload.get("filename"),
            "chunk_index": payload.get("chunk_index")
        })
    return contexts

def build_prompt(contexts, user_instruction):
    ctx_texts = "\n\n---\n\n".join([
        f"Source: {c['filename']} (chunk {c['chunk_index']}):\n{c['text']}"
    for c in contexts
    ])
    prompt = f"""You are a contract drafting assistant. Use the following
source contract excerpts to answer or generate a contract per the user's
instruction.

Context from uploaded contracts:
{ctx_texts}
```

```
User instruction:
{user_instruction}
```

Requirements:

- Preserve legal clarity.
- When information is missing, indicate placeholders like <Insert X>.
- Output clean contract text with headings and sections.

```
Answer: ""
```

```
    return prompt
```

```
def call_ollama(prompt, max_tokens=1500, temperature=0.1):
```

```
    payload = {
```

```
        "model": OLLAMA_MODEL,
```

```
        "prompt": prompt,
```

```
        "max_tokens": max_tokens,
```

```
        "temperature": temperature,
```

```
        "stream": False
```

```
    }
```

```
    r = requests.post(OLLAMA_API, json=payload, timeout=120)
```

```
    r.raise_for_status()
```

```
    return r.json()
```

```
def qa_flow(query: str):
```

```
    contexts = retrieve_contexts(query, top_k=6)
```

```
    prompt = build_prompt(contexts,
```

```
f"Answer this question concisely based on the contracts: {query}")
```

```
    resp = call_ollama(prompt)
```

```
    return {"contexts": contexts, "ollama": resp}
```

```
def generate_contract_flow(instruction: str):
```

```
    contexts = retrieve_contexts(instruction, top_k=8)
```

```
    prompt = build_prompt(contexts, instruction)
```

```
    resp = call_ollama(prompt, max_tokens=2500)
```

```
    return {"contexts": contexts, "ollama": resp}
```

backend/app.py

```
# app.py
```

```
import os
```

```
import shutil
```

```
from fastapi import FastAPI, UploadFile, File, Form
```

```
from fastapi.responses import JSONResponse, FileResponse
```

```
from pathlib import Path
```

```
import uvicorn
```

```

from ingest_contracts import ingest_folder
from rag import qa_flow, generate_contract_flow

UPLOAD_DIR = Path("uploaded_contracts")
UPLOAD_DIR.mkdir(exist_ok=True)

app = FastAPI()

@app.post("/upload")
async def upload_contracts(files: list[UploadFile] = File(...)):
    saved = []
    # limit to 5 files
    if len(files) > 5:
        return JSONResponse({"error": "Max 5 files allowed."},
status_code=400)
    for f in files:
        dest = UPLOAD_DIR / f.filename
        with open(dest, "wb") as out:
            shutil.copyfileobj(f.file, out)
        saved.append(str(dest))
    return {"saved": saved}

@app.post("/ingest")
async def ingest():
    cnt = ingest_folder(str(UPLOAD_DIR))
    return {"status": "ingested", "vectors_upserted": cnt}

@app.post("/qa")
async def qa(query: str = Form(...)):
    result = qa_flow(query)
    return JSONResponse(content=result)

@app.post("/generate")
async def generate(instruction: str = Form(...)):
    result = generate_contract_flow(instruction)
    # optionally convert to docx and return a file; for now return ollama
output
    return JSONResponse(content=result)

if __name__ == "__main__":
    uvicorn.run(app, host="0.0.0.0", port=8001)

```

frontend/package.json

```
{
  "name": "rag-contract-frontend",
  "version": "0.1.0",
  "private": true,
  "dependencies": {
    "axios": "^1.4.0",
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "react-scripts": "5.0.1"
  },
  "scripts": {
    "start": "react-scripts start",
    "build": "react-scripts build"
  }
}
```

frontend/src/GenerateContract.jsx

```
// GenerateContract.jsx
import React, {useState} from "react";
import axios from "axios";

export default function GenerateContract(){
  const [files, setFiles] = useState(null);
  const [status, setStatus] = useState("");
  const [qaQuery, setQaQuery] = useState("");
  const [qaResult, setQaResult] = useState(null);
  const [instr, setInstr] = useState("");
  const [genResult, setGenResult] = useState(null);

  const handleFiles = (e) => {
    setFiles(e.target.files);
  }

  const uploadAndIngest = async () => {
    if(!files) return alert("Choose files first (max 5)");
    const form = new FormData();
    for(let i=0;i<files.length && i<5;i++){
      form.append('files', files[i]);
    }
    setStatus('Uploading...')
    await axios.post('http://localhost:8001/upload', form);
    setStatus('Ingesting...')
    await axios.post('http://localhost:8001/ingest');
    setStatus('Ready');
  }
}
```

```

}

const runQA = async () => {
  setStatus('Running QA...')
  const resp = await axios.post('http://localhost:8001/qa', new
URLSearchParams({query: qaQuery}));
  setQaResult(resp.data);
  setStatus('Ready');
}

const runGenerate = async () => {
  setStatus('Generating...')
  const resp = await axios.post('http://localhost:8001/generate', new
URLSearchParams({instruction: instr}));
  setGenResult(resp.data);
  setStatus('Ready');
}

return (
  <div style={{padding:20, fontFamily:'sans-serif'}}>
    <h2>Generate Contract (RAG + Local Mistral)</h2>
    <div>
      <input type="file" multiple accept=".pdf,.docx"
onChange={handleFiles} />
      <div>Upload up to 5 contracts (PDF or DOCX)</div>
      <button onClick={uploadAndIngest} style={{marginTop:8}}>Upload &
Ingest</button>
    </div>

    <hr />

    <div>
      <h3>Question Answering</h3>
      <textarea rows={3} cols={80} value={qaQuery}
onChange={(e)=>setQaQuery(e.target.value)} placeholder="Ask a question about
uploaded contracts" />
      <br/>
      <button onClick={runQA}>Run QA</button>
      <pre style={{whiteSpace:'pre-wrap', background:'#f4f4f4', padding:
10}}>{qaResult? JSON.stringify(qaResult, null, 2) : ''}</pre>
    </div>

    <hr />

    <div>
      <h3>Generate Contract</h3>
      <textarea rows={6} cols={80} value={instr}
onChange={(e)=>setInstr(e.target.value)}
placeholder={`E.g. Generate contract similar to uploaded ones but: 1) First
Party PrimeNumerics; 2) Second Party Infosys; 3) Duration: 2026-01-01 to
2027-01-01; 4) Governing Law: New York`} />

```



```

        <br />
        <button onClick={runGenerate}>Generate Contract</button>
        <pre style={{whiteSpace:'pre-wrap', background:'#f4f4f4', padding:
10}}>{genResult? JSON.stringify(genResult, null, 2) : ''}</pre>
        </div>

        <div style={{marginTop:12}}>Status: {status}</div>
    </div>
)
}

```

README.md

```

# RAG-Contract-Demo (Local)

## Overview
This demo ingests up to 5 contracts (PDF/DOCX), stores chunk embeddings in
Qdrant, and uses a local Mistral model via Ollama to do QA and to generate a
new contract using RAG.

## Prerequisites
- Docker (for Qdrant)
- Ollama installed (https://ollama.com) and a Mistral model pulled: `ollama
pull mistral` (confirm exact model name with `ollama list`)
- Python 3.10+
- Node.js (for frontend)

## Quick start
1. Start Qdrant:
    ```bash
 docker run -d --name qdrant -p 6333:6333 -v qdrant_storage:/qdrant/storage
qdrant/qdrant
    ```

2. Ensure Ollama is running and model is pulled:
    ```bash
 ollama pull mistral
 # run ollama (the installer usually runs a local API automatically)
 ollama run mistral
    ```

3. Backend:
    ```bash
 cd backend
 python -m venv .venv
 source .venv/bin/activate # or .venv\Scripts\activate on Windows
 pip install -r requirements.txt
 uvicorn app:app --reload --port 8001
    ```

```

```
'''

4. Frontend:
```bash
cd frontend
npm install
npm start
```

5. Use the UI:
- Upload up to 5 files, click `Upload & Ingest`.
- Ask questions in QA, or enter a generate instruction and click `Generate Contract`.

## Notes
- The Ollama API default endpoint used is `http://localhost:11434/api/generate`. If your Ollama installation exposes a different endpoint or model name, set environment variable `OLLAMA_API` and `OLLAMA_MODEL` before running backend.
- This project is a demo. Any generated contract must be reviewed by a human lawyer before use.
```

What I did: I created a complete, runnable local demo project (backend + frontend + docs) in this canvas. Open each file on the canvas to copy or download.

If you'd like, next I can: - produce the `.docx` export endpoint to download generated contracts, or - package the backend into a single Docker Compose for Qdrant + backend + optional Ollama proxy, - or adapt this to your existing backend structure and provide diff/patches.

Which of these should I do next? (I can start immediately.)